CS/ECE 4501 / CS 6501 / ECE 6782 **Due by Sep 16th** 11 : 59 : 59**PM**

Problem Set I

1. (20%) Prove the properties of convolution. For all continuous function f, g, and h, the following axioms hold. **Make sure to lay out all key steps of your proof for full credits.**

   - Associativity: $(f * g) * h = f * (g * h)$
   - Distributivity: $f * (g + h) = f * g + f * h$
   - Differentiation rule: $(f * g)' = f' * g = f * g'$
   - Convolution theorem: $\mathcal{F}(g * h) = \mathcal{F}(g)\mathcal{F}(h)$, where $\mathcal{F}$ denotes Fourier transform

2. (25%) Frequency smoothing:

   (a) Compute Fourier transform of the given image `lenaNoise.PNG` by using `numpy.fft.fft2` in Python, and then center the low frequencies (e.g., by using `fftshift`).

   (b) Keep different number of low frequencies (e.g., $7^2, 15^2, 31^2$ and the full dimension), but set all other high frequencies to 0.

   (c) Reconstruct the original image (`ifft2`) by using the new generated frequencies in step (b).

   Submit the code and include the restored images with different number of low frequencies in your report.

3. (55%) Implement the gradient descent (GD) algorithm for image denoising using the Total Variation (ROF) model discussed in class. You should implement the GD optimization yourself and avoid using different optimization schemes found online. A detailed class note of deriving total variation, computing gradient term, and gradient descent algorithm can be downloaded from Canvas.

   All executable code and a report must be submitted, including the problem description, the optimization objective function, and experimental results (denoised images and with convergence graph generated using your best-tuned parameters), along with discussions of your findings.

   - With the given image `Einstein.jpeg`, generate different noisy images with additive Gaussian noises at different variance ($\sigma = 0.01, 0.05, 0.1$).
   - Use central difference scheme to compute image gradient. You may use a python library (e.g., `numpy.gradient`) to compute the image gradient, but you should implement your own divergence operator using the gradient results.
   - Run your denoising algorithm on the generated noisy images. Note that you may need to manually tune the weighting parameter λ and the learning rate/step size in gradient descent to achieve the best performance. Please report only the results with your model's best performance.
   - Plot a convergence graph reporting the objective function $E(u^k)$ over all iterations.